One application through GitHub Actions

DevOps & infrastructure | Intermediate

Session M4-01 | 3 hours

Agenda

Time	Topic
0:00 to 0:15	Workflow anatomy
0:15 to 0:23	Reusable workflows, custom actions, common mistakes
0:23 to 0:35	Generate and fix CI with Copilot and `actionlint`
0:35 to 0:45	Design deployment
0:45 to 0:55	Repair workflow failures
0:55 to 1:00	Review and lab handoff

Anatomy of a workflow

Part	What it is
Workflow	A YAML file in `.github/workflows/`
Trigger	The `on:` block. Decides who can start a run
Job	Steps on one runner. Parallel unless `needs:` orders them
Runner	A fresh machine, such as `ubuntu-latest`
Step	One `run:` command or one `uses:` action
Action	Reusable code a step calls

Anatomy: data, access, and gates

Part	What it is
Artifact	Files a job uploads. Jobs don't share a disk
`needs`	Job order. A failed job skips its dependents
Permissions	What the job's `GITHUB_TOKEN` can do. Set the minimum
Secrets	Stored values, read as `${{ secrets.NAME }}`
Environment	A named target such as `production`. Holds secrets and can require reviewers

The environment's required reviewers are the deployment gate.

Start with repository facts

`copilot-webapp` already defines the contract:

```
npm ci
npm test
npm run lint
npm run build
test -f dist/app.js
```

The workflow calls these commands. It doesn't invent another build, artifact path, or runtime.

When to reuse

Option	Use it when
Reusable workflow (`workflow_call`)	Several repositories run the same pipeline
Composite or JavaScript action (`action.yml`)	One step needs real code, or you want to test it locally
Neither	One repository, one pipeline

An action that never fails hasn't shown it enforces anything. Test a passing and a failing run.

Common YAML mistakes

- Indentation that changes what counts as a step.
- Typos that stay valid YAML: `refs/head/main` never matches.
- Old runner labels and action versions.
- No `actions/checkout`, so the job has no code.
- A secret name that doesn't match the stored one.
- No `permissions:` block.
- Secrets used directly in a step's `if:`.

A linter finds some of these. You find the rest by reading the spec.

Generate CI from the application

```
Read package.json, tests, and scripts/build.js. Create ci.yml for pull requests
and pushes to main. Use Node.js 20, npm ci, existing lint, test, and build scripts,
npm caching, contents: read, and a seven-day dist artifact.
```

Check the draft with `actionlint` and the application's own commands:

```
actionlint ci.yml
npm test && npm run lint && npm run build
```

Copilot edits, the linter decides

In Copilot CLI (`copilot`) or agent mode:

1. Prompt: generate `ci.yml` from the repository.
2. Run `actionlint ci.yml` yourself.
3. Paste the exact message back: "Fix this with the smallest change."
4. Rerun `actionlint` and the npm commands.

Example: `runs-on: ubuntu-18.04` gives an "unknown label" message on one line. The fix changes that line.

Deployment stays a design exercise

```
build → staging → production
```

The workflow references environments and secrets. Production reviewers are set in repository settings. Placeholder commands must not imply a deployment ran.

Syntax and behavior fail differently

`broken-ci.yml` doesn't parse. `actionlint` points at `steps:`, but the cause is the step list below it: the first step is indented less than the rest.

`broken-deploy.yml` parses. `actionlint` reports only the stale runner. The bad Git ref, the wrong secret name, and the missing checkout need the spec and a careful read.

Get the output before the repair.

Tie each repair to a failure

```
Observation: the parser fails near Setup Node.js.
Change: align the step indentation.
Proof: parse the file again.
```

For behavior faults, cite the deployment spec or repository facts, then check the repair.

Lab

Generate and lint CI, design deployment, and repair failures on `copilot-webapp`. The custom action is optional.